Белорусский государственный технологический университет
Факультет информационных технологий
Кафедра программной инженерии

ОТЧЕТ

по лабораторной работе № 4
по дисциплине «Операционные системы»
Управление потоками

Выполнил:
Студент 3 курса 9 группы
Антипов Алексей Романович
Преподаватель: Уласевич Н. И.

Содержание

Введение ... 3
1 Теоретические основы управления потоками ... 4
1.1 Потоки и ожидание завершения ... 4
1.2 Локальное хранение данных потока ... 4
1.3 Волокна и пользовательские контексты ... 4
2 Реализация приложений Windows ... 5
2.1 Функция Lab_04x ... 5
2.2 Приложение Lab 04a ... 6
2.3 Приостановка и возобновление в Lab 04b ... 9
2.4 Принудительное завершение в Lab 04c ... 13
2.5 Перечисление потоков в Lab 04d ... 16
2.6 Поиск простых чисел и WinAPI TLS ... 17
2.7 Параметризованный поиск в Lab 04p ... 18
2.8 Статический TLS в Lab 04p2 ... 19
3 Сравнение производительности ... 20
3.1 Условия проведения измерений ... 20
3.2 Фиксация результатов и интерпретация ... 22
4 Вычисление числа π с использованием волокон ... 24
4.1 Приложение Lab 04pi в Windows ... 24
4.2 Выбор размера частичной суммы ... 25
5 Реализация приложений Linux ... 27
5.1 Создание потоков и функция Lab_04x ... 27
5.2 Локальное хранение в Lab 04p Linux ... 28
5.3 Вычисление π через пользовательские контексты ... 32
Заключение ... 35
Список использованных источников ... 36
Приложение А Основные фрагменты программ ... 37
Локальные данные потоков Windows и Linux ... 38
Расчёт блока ряда в волокне Windows ... 40
Создание и переключение контекстов Linux ... 41

Введение

Цель работы — получение практических навыков создания потоков и управления их выполнением в операционных системах Windows и Linux. Рассматриваются ожидание завершения потоков, приостановка и возобновление выполнения, локальное хранение данных и кооперативное переключение вычислительных контекстов.

В Windows подготовлены приложения Lab-04a, Lab-04b, Lab-04c и Lab-04d, варианты поиска простых чисел Lab-04e, Lab-04p и Lab-04p2, а также приложение Lab-04pi. В Linux подготовлены Lab-04px, Lab-04p_Linux и Lab-04pi_Linux. Общая функция Lab_04x выводит идентификаторы процесса и потока, номер итерации и циклически выбранные символы имени пользователя и ФИО.

Для достижения цели работы решаются следующие задачи:

– создание двух дочерних потоков и ожидание их завершения главным потоком;

– наблюдение за потоками средствами PowerShell, Process Explorer, ps и файловой системы /proc;

– исследование изменения состояния потоков при приостановке, возобновлении и принудительном завершении;

– поиск простых чисел с использованием TLS и анализ совместной записи в общий массив;

– сравнение времени выполнения при различном числе вычислительных потоков;

– вычисление приближения числа π с помощью волокон и пользовательских контекстов.

Результаты работы приложений представляются снимками консоли и средствами мониторинга. Отдельно фиксируются показатели производительности, корректность полученных данных и освобождение ресурсов на штатном пути выполнения.

1 Теоретические основы управления потоками

1.1 Потоки и ожидание завершения

Поток представляет собой последовательность выполнения команд внутри процесса. Потоки одного процесса используют общее адресное пространство, но имеют собственный стек и контекст выполнения. Поэтому одинаковый PID при различных TID соответствует нескольким потокам одного процесса. Планировщик может чередовать их выполнение или распределять потоки по различным ядрам.

В Windows поток создаётся функцией CreateThread. Дескриптор потока позволяет ожидать его завершения и управлять выполнением. В Linux соответствующие операции выполняются с помощью pthread_create и pthread_join. Ожидание завершения требуется перед освобождением данных, переданных работающим потокам.

1.2 Локальное хранение данных потока

Thread Local Storage обеспечивает отдельное значение переменной или указателя для каждого потока. В Windows динамический TLS использует TlsAlloc, TlsSetValue, TlsGetValue и TlsFree [1]. Вариант __declspec(thread) задаёт потоковую область хранения при объявлении переменной. В Linux применяются ключи pthread_key_t и модификатор __thread.

TLS разделяет промежуточные данные, но не защищает общий массив и общий индекс записи. Если несколько потоков изменяют один счётчик без синхронизации, результат зависит от порядка выполнения операций. Сортировка после ожидания потоков не восстанавливает значения, потерянные при записи.

1.3 Волокна и пользовательские контексты

Волокна переключаются по инициативе программы. В Windows для этого используются ConvertThreadToFiber, CreateFiber и SwitchToFiber [4]. В реализации Linux контекст задаётся структурой ucontext_t, а переключение выполняется swapcontext [8]. Несколько волокон внутри одного потока не обеспечивают одновременное вычисление на нескольких ядрах.

Кооперативное выполнение позволяет делить длительную работу на небольшие порции. Размер порции определяет частоту переключений, а проверка условия завершения позволяет остановить вычисление до окончания очередной порции.

2 Реализация приложений Windows

2.1 Функция Lab_04x

Функция Lab_04x получает количество итераций через аргумент. Для имени пользователя и ФИО выделяются массивы wchar_t. В них записаны строки «User-3af11550» и «Антипов Алексей Романович». Выбор символов выполняется по остатку от деления номера итерации на длину соответствующей строки.

PID и TID определяются функциями GetCurrentProcessId и GetCurrentThreadId. Вывод организован через wprintf из семейства форматированного вывода printf. Формат строки содержит PID, TID, номер итерации, символ имени пользователя и символ ФИО. Задержка Sleep изменяется от 350 до 450 мс.

В отдельном тестовом приложении функция Lab_04x вызывается с аргументом 30. На рисунке 2.1 показаны результаты выполнения 30 итераций. На 14-й итерации последовательность символов имени пользователя повторяется с первого символа, а обход строки ФИО продолжается независимо.

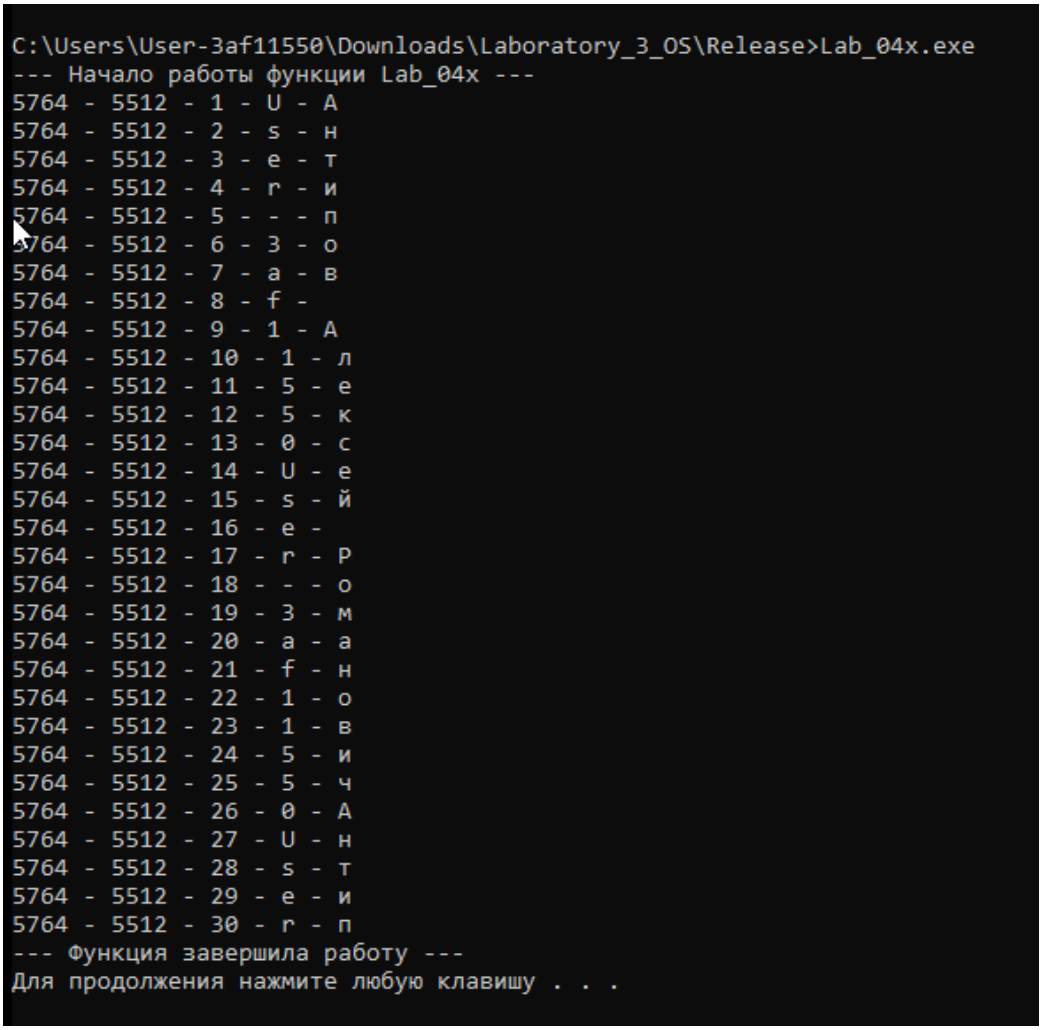


Рисунок 2.1 – Циклический вывод функции Lab_04x

На первой итерации выбираются символы «U» и «А». После окончания строки имени пользователя её символы повторяются, а обход ФИО продолжается независимо. После штатного завершения цикла оба динамических массива освобождаются вызовами free.

2.2 Приложение Lab 04a

В приложении Lab-04a функция CreateThread создаёт два дочерних потока. Функция ThreadWrapper приводит переданный аргумент к типу int* и разыменовывает указатель, получая число итераций для вызова Lab_04x. Первый дочерний поток выполняет 55 итераций, второй — 130, главный поток — 100. Фрагменты реализации приведены в приложении А.

По окончании собственного цикла главный поток вызывает WaitForMultipleObjects с параметрами TRUE и INFINITE. Ожидание завершается после окончания обоих дочерних потоков. Затем их дескрипторы закрываются функцией CloseHandle. Вывод Lab-04a сохранён в файл Lab-04a-output.txt. Средствами PowerShell строки журнала сгруппированы по TID; для каждого потока определены количество строк и максимальный номер итерации. Результаты проверки представлены на рисунке 2.2.

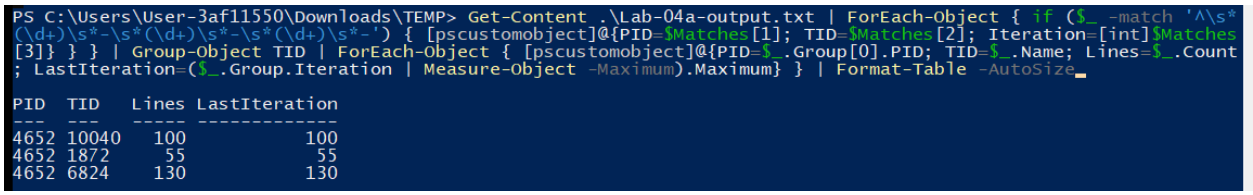


Рисунок 2.2 – Выполнение трёх потоков в Lab 04a

Для процесса PID 4652 главный поток TID 10040 выполнил 100 итераций, первый дочерний поток TID 1872 — 55, второй дочерний поток TID 6824 — 130. Для каждого потока количество строк совпадает с максимальным номером итерации. Результаты соответствуют заданным количествам повторений.

Порядок строк разных потоков не задаётся программой и зависит от планирования. Совпадение PID показывает, что вычисления выполняются в одном процессе.

Во время работы приложения в PowerShell выполнена команда (Get-Process -Id 2068).Threads | Select-Object Id, ThreadState. На рисунке 2.3 для процесса PID 2068 сопоставлены идентификаторы потоков в консоли, PowerShell и вкладке Threads окна свойств процесса в Process Explorer. В трёх источниках присутствуют TID 9240, 9056 и 4836.

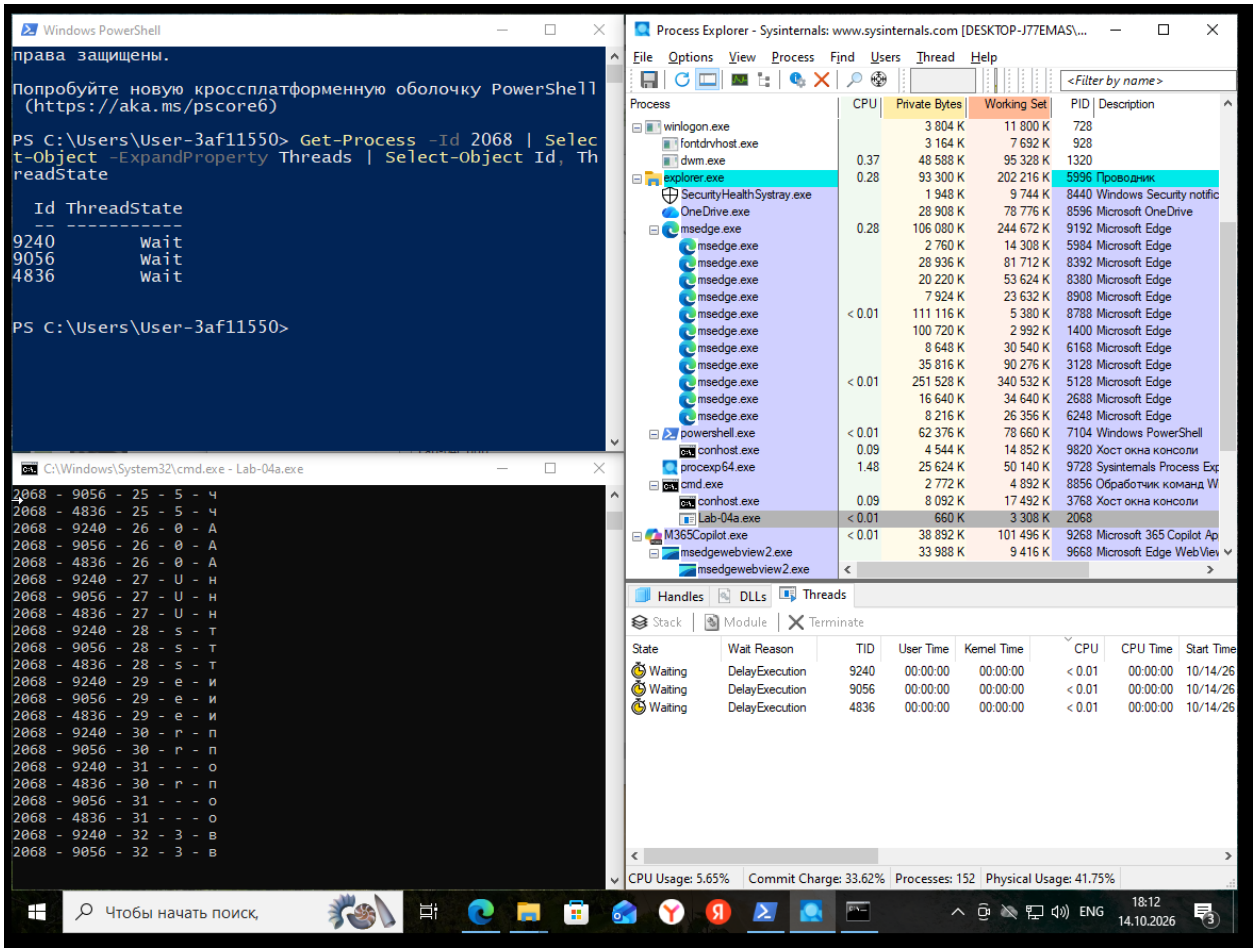


Рисунок 2.3 – Потоки Lab 04a в средствах мониторинга

Во время Sleep поток ожидает истечения задержки. Помимо потоков, созданных в исходном коде, в процессе могут присутствовать служебные потоки библиотек. Поэтому сравнение выполняется по TID, а не только по общему количеству строк.

2.3 Приостановка и возобновление в Lab 04b

В Lab-04b главный поток выполняет собственный цикл из 100 итераций. Условия i == 29, i == 39 и i == 69 соответствуют 30-й, 40-й и 70-й итерациям. Этапы управления дочерними потоками приведены в таблице 2.1.

Таблица 2.1 – Этапы управления потоками Lab 04b

Момент главного цикла	**Действие главного потока**	**Первый дочерний поток**	**Второй дочерний поток**
До 30-й итерации	Выполнение цикла	Не приостановлен	Не приостановлен
30-я итерация	SuspendThread для первого потока	Приостановлен	Не приостановлен
40-я итерация	SuspendThread для второго потока	Приостановлен	Приостановлен
70-я итерация	ResumeThread для первого потока	Возобновлён	Приостановлен
После завершения 100 итераций	ResumeThread для второго потока, затем ожидание обоих	Выполняется либо уже завершён	Возобновлён

После 40-й итерации оба дочерних потока приостановлены. Первый получает возможность продолжить выполнение на 70-й итерации, а второй — после окончания главного цикла.

На рисунке 2.4 фиксируется этап после 40-й и до 70-й итерации главного потока, когда оба дочерних потока приостановлены.

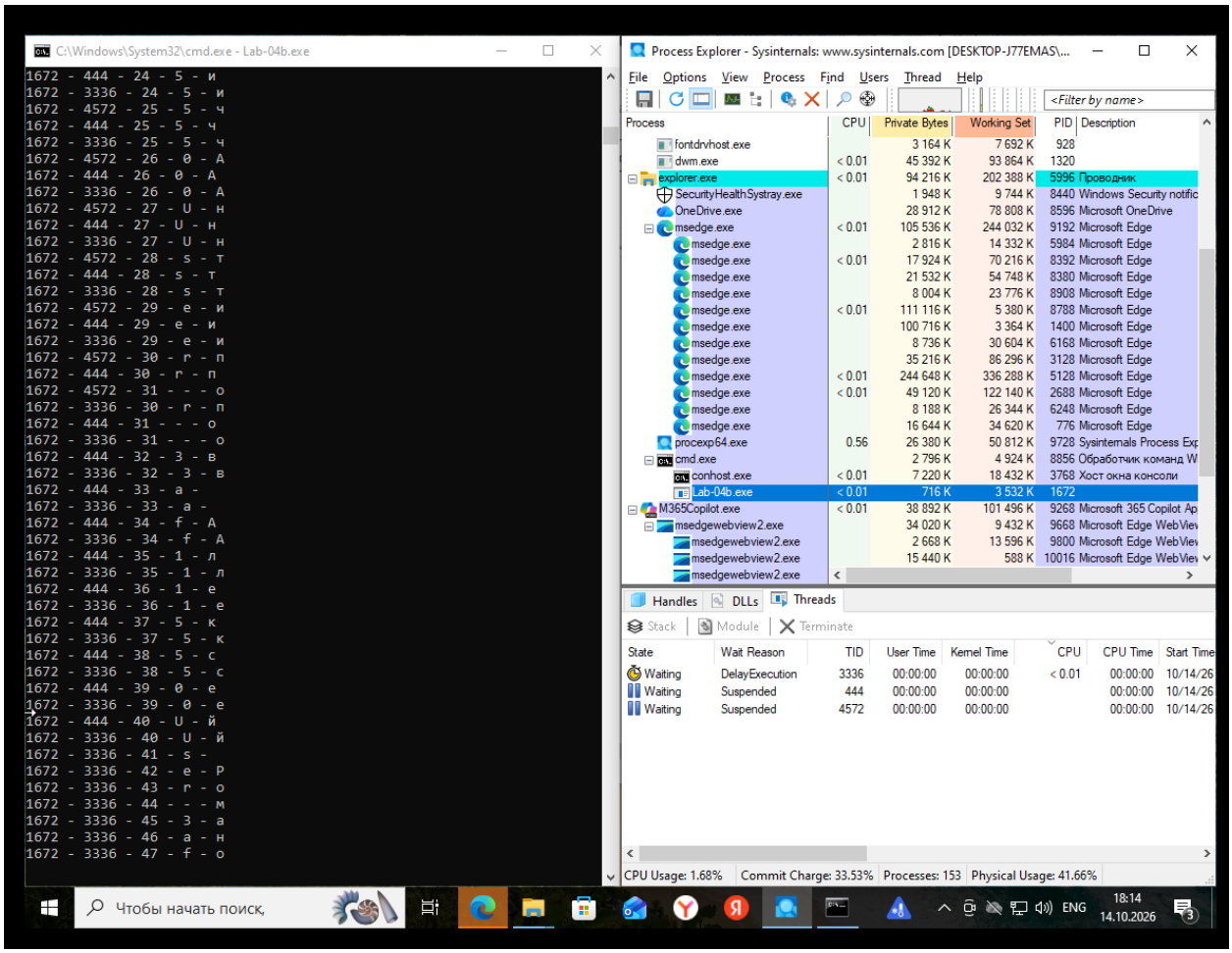


Рисунок 2.4 – Приостановленные дочерние потоки Lab 04b

Приостановленный поток не продолжает свой цикл после окончания Sleep. SuspendThread увеличивает счётчик приостановки; вызов ResumeThread уменьшает его [2].

На рисунке 2.5 зафиксирован первый из двух рассматриваемых этапов: после 70-й итерации главного потока первый дочерний поток возобновлён, а второй всё ещё приостановлен. Состояние каждого рабочего потока определяется по его TID.

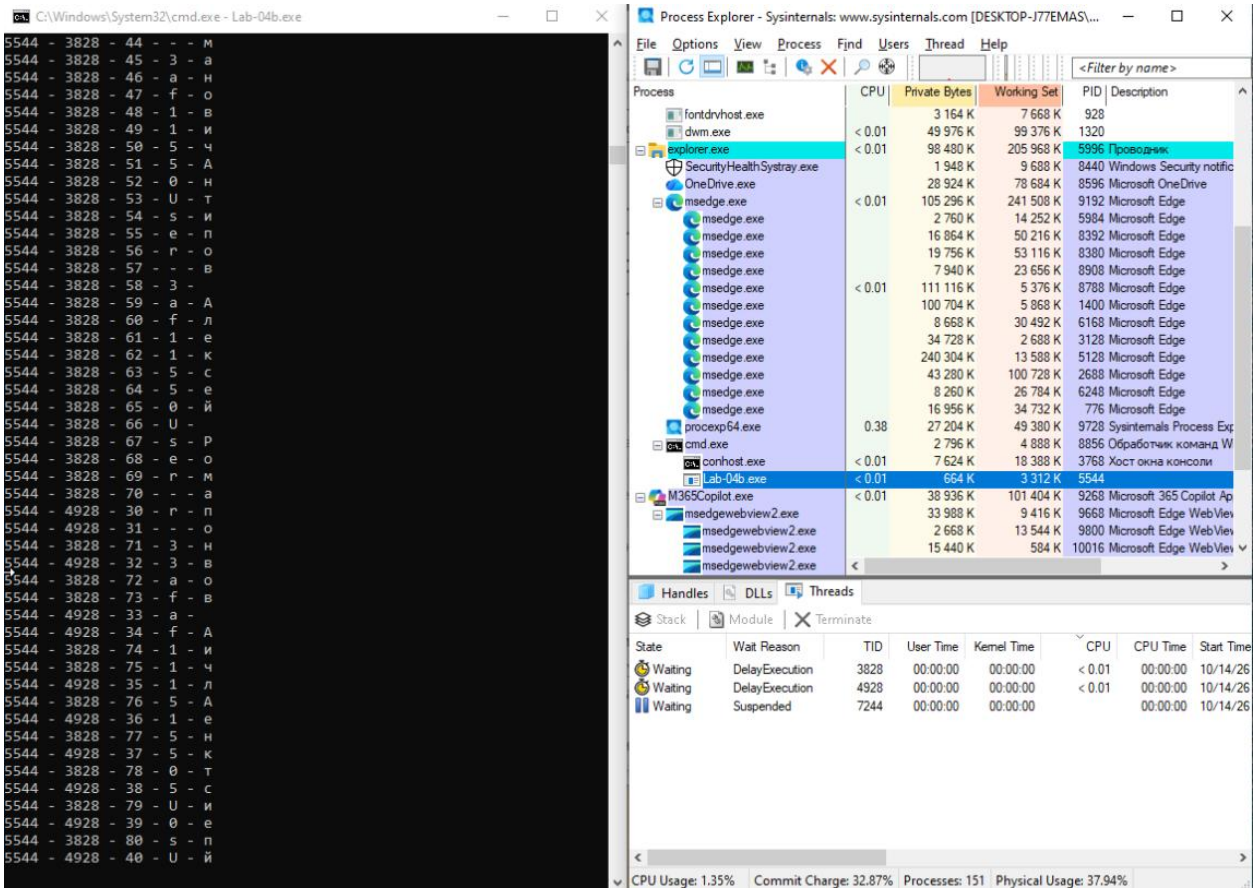


Рисунок 2.5 – Возобновление первого дочернего потока Lab-04b

На рисунке 2.5 во вкладке Threads окна свойств процесса в Process Explorer первый дочерний поток находится в состоянии Waiting с причиной ожидания DelayExecution. Это соответствует выполнению задержки Sleep после возобновления потока. Второй дочерний поток остаётся в состоянии Suspended.

После завершения главного цикла из 100 итераций вызывается ResumeThread для второго дочернего потока. При достижении нулевого значения счётчика приостановки поток получает возможность продолжить выполнение. Затем главный поток ожидает завершения обоих дочерних потоков. Возобновление вывода второго дочернего потока показано на рисунке 2.6.

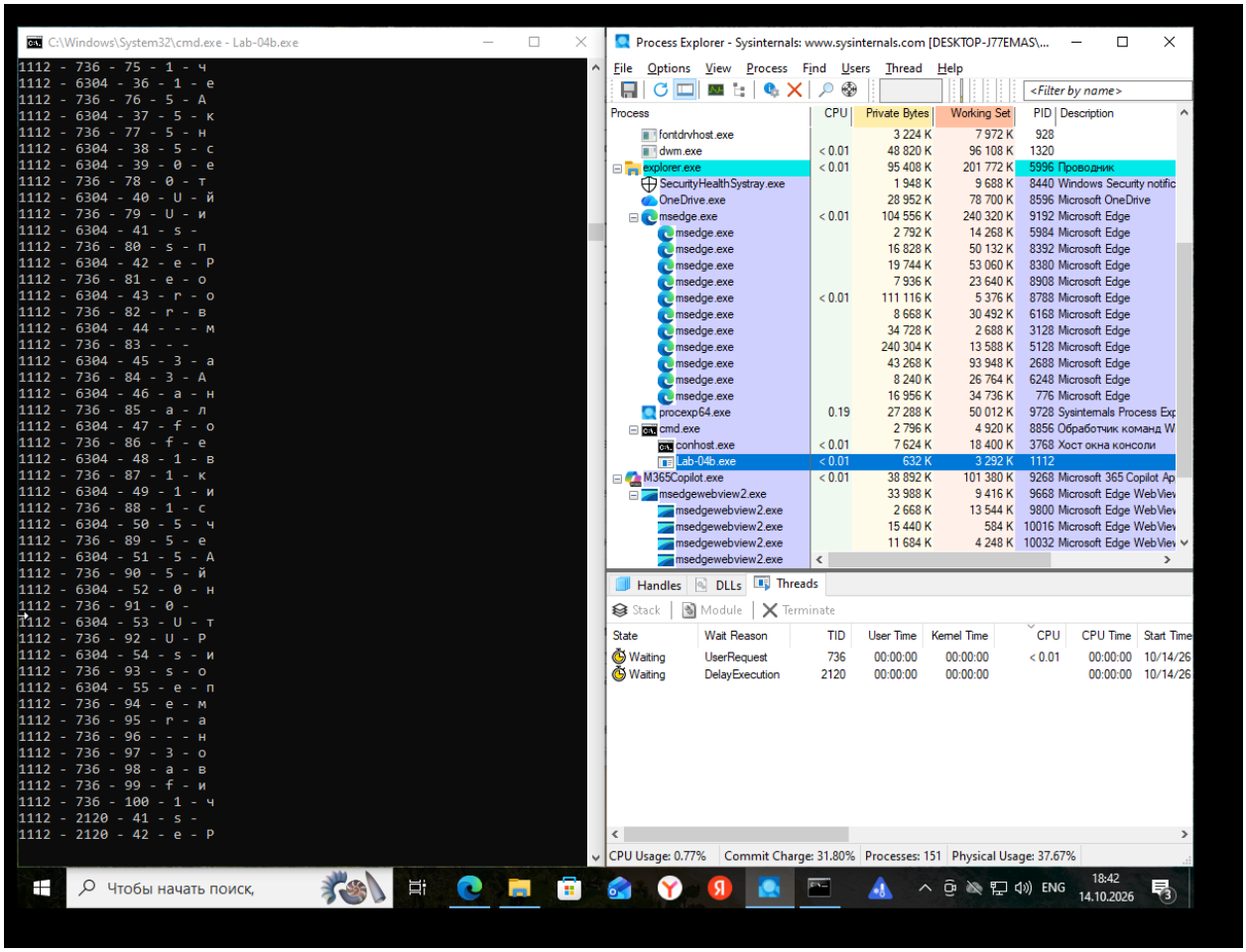


Рисунок 2.6 – Возобновление второго дочернего потока Lab 04b

На рисунке 2.6 главный поток достиг 100-й итерации, а второй дочерний поток продолжил вывод с 41-й итерации. Первый дочерний поток к моменту снимка уже завершился, поэтому в списке Threads остаются главный и второй дочерний потоки. Приостановка потока, удерживающего внутреннюю блокировку библиотеки, может задержать выполнение других потоков.

2.4 Принудительное завершение в Lab 04c

В приложении Lab-04c на 50-й итерации главного потока вызывается TerminateThread для второго дочернего потока. В показанном запуске процесс имеет PID 7856, главный поток — TID 10548, первый дочерний — TID 8676, второй дочерний — TID 9752. На рисунке 2.7 все три идентификатора присутствуют в списке Threads приложения Process Explorer. В момент снимка потоки находятся в состоянии Waiting с причиной ожидания DelayExecution, соответствующей задержке Sleep.

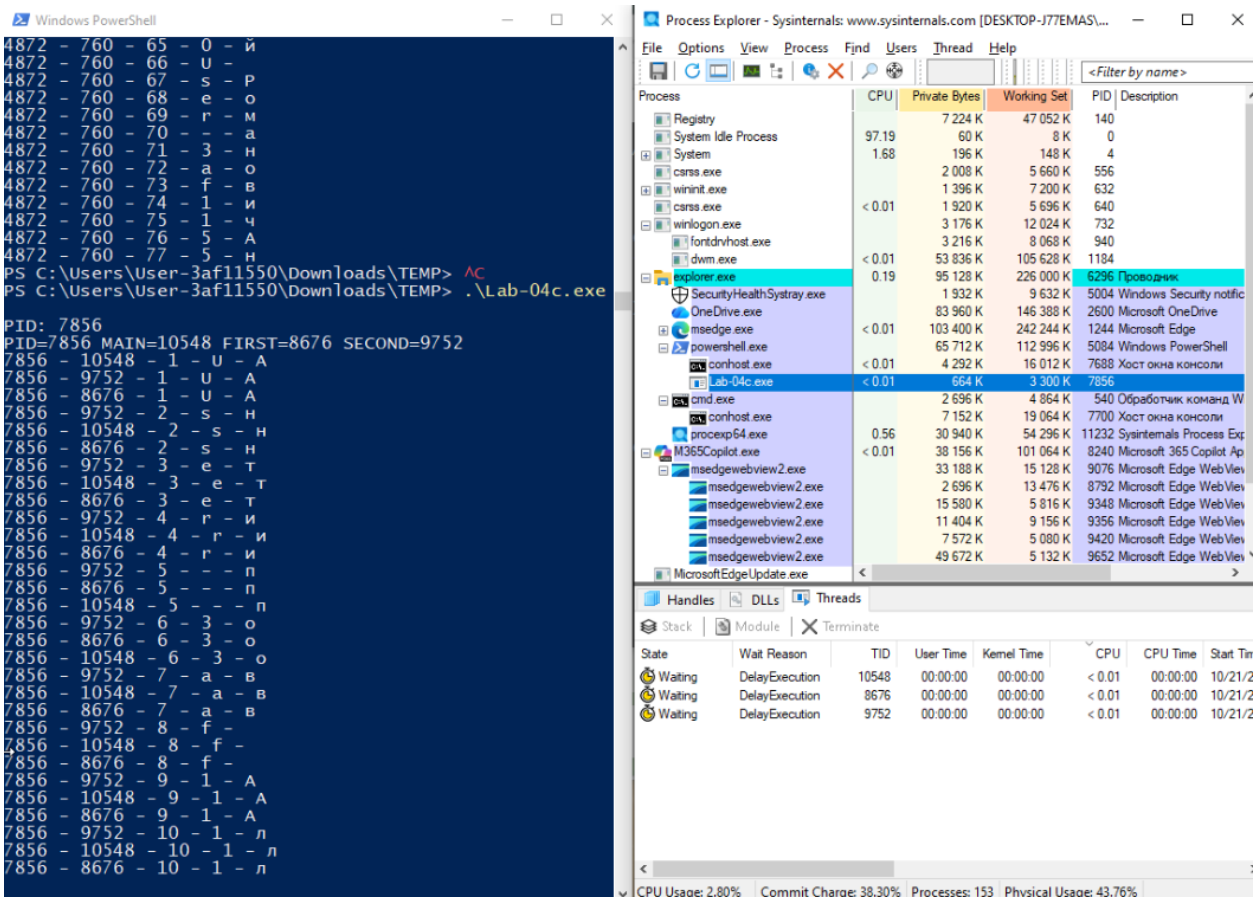


Рисунок 2.7 – Потоки Lab-04c до принудительного завершения второго дочернего потока

На 50-й итерации главного потока выполнен успешный вызов TerminateThread для TID 9752. Завершение подтверждено ожиданием дескриптора через WaitForSingleObject и получением кода завершения 0 функцией GetExitCodeThread. На рисунке 2.8 видна строка подтверждения, а TID 9752 отсутствует в обновлённом списке Threads. Главный и первый дочерний потоки продолжают вывод после завершения второго дочернего потока.

Состояние потоков после принудительного завершения второго дочернего потока показано на рисунке 2.8.

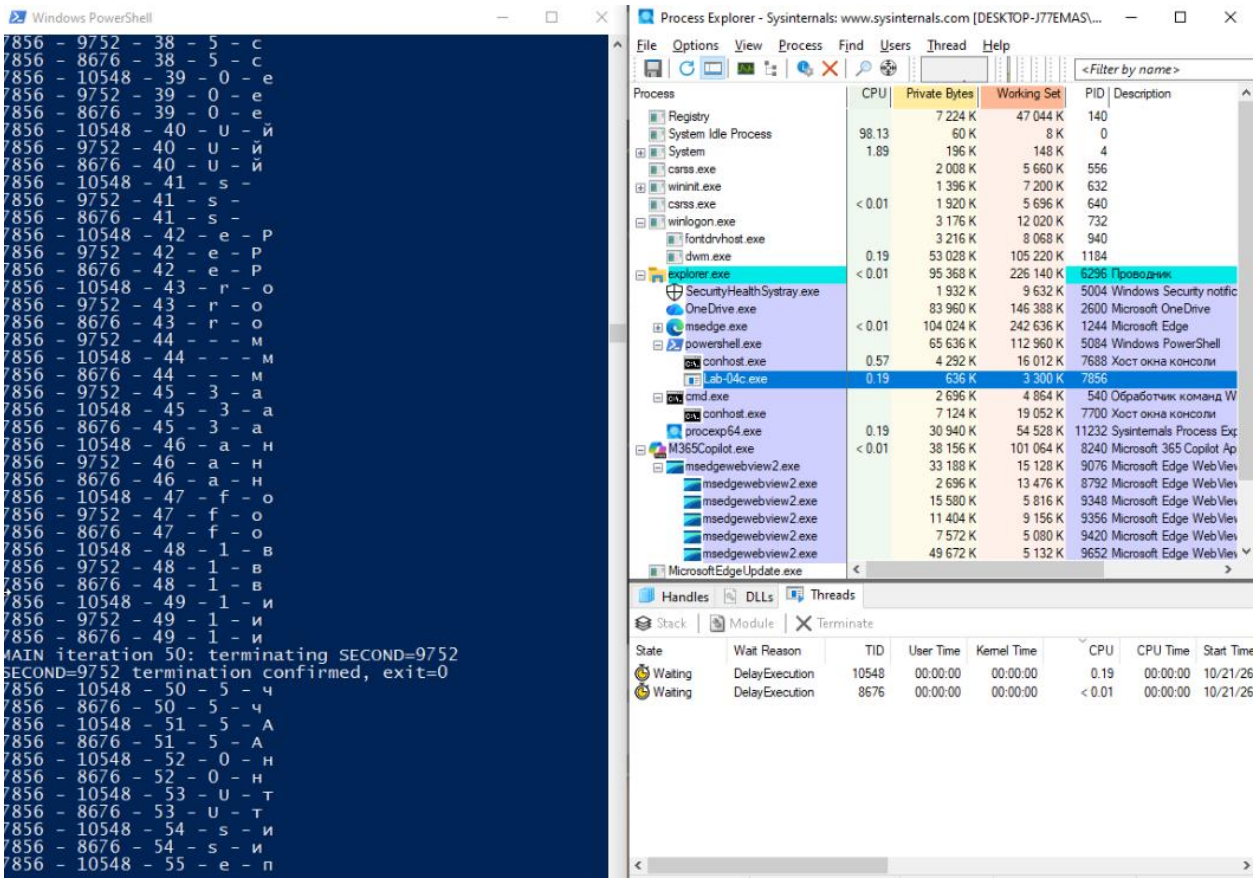


Рисунок 2.8 – Потоки Lab-04c после завершения второго дочернего потока

В данном запуске оставшиеся потоки продолжили выполнение, однако это не гарантирует безопасность TerminateThread. Принудительно завершаемый поток не выполняет предусмотренную программой очистку и может оставить занятыми блокировки общих ресурсов. После окончания главного цикла приложение ожидает завершения дочерних потоков через WaitForMultipleObjects, затем закрывает их дескрипторы. Отсутствие TID в списке Threads само по себе не означает закрытие дескриптора [3].

Для проверки сохранения дескрипторов выполнен отдельный запуск Lab-04c с PID 10636. Главный поток имеет TID 10752, первый дочерний — TID 1680, второй дочерний — TID 9264. На рисунке 2.9 в панели Handles приложения Process Explorer показаны открытые дескрипторы объектов Thread для обоих дочерних потоков до выполнения 50-й итерации главного потока.

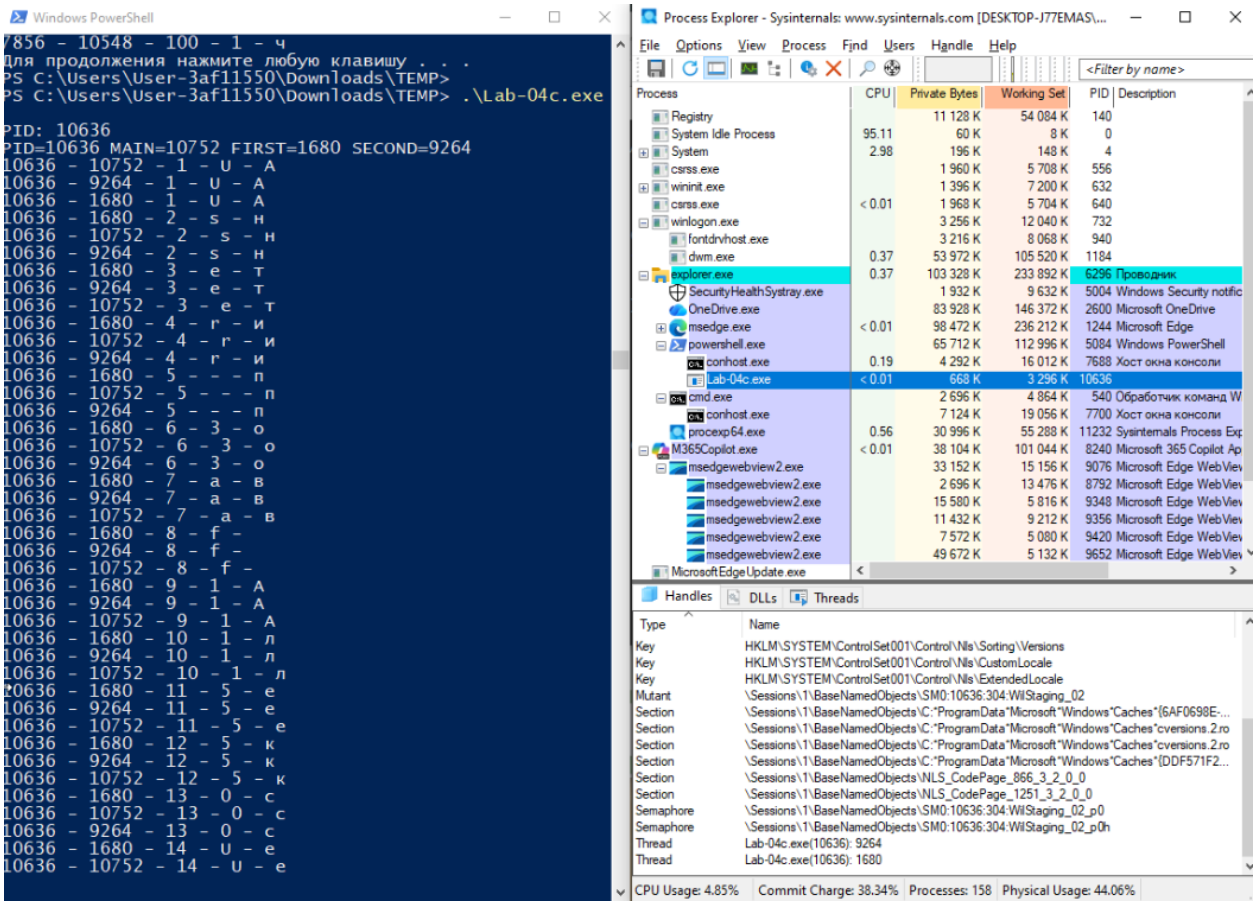


Рисунок 2.9 – Дескрипторы дочерних потоков Lab-04c до принудительного завершения

Идентификаторы 1680 и 9264 в именах объектов Thread совпадают с TID дочерних потоков, указанными в консоли. Эти номера являются идентификаторами потоков, а не значениями дескрипторов. Наличие открытого дескриптора позволяет обращаться к объекту потока, но само по себе не подтверждает продолжение его выполнения.

На 50-й итерации главный поток завершает второй дочерний поток TID 9264 вызовом TerminateThread. Завершение подтверждено ожиданием дескриптора и получением кода завершения 0. На рисунке 2.10 одновременно показаны строка подтверждения в консоли и сохранившийся дескриптор соответствующего объекта Thread в панели Handles.

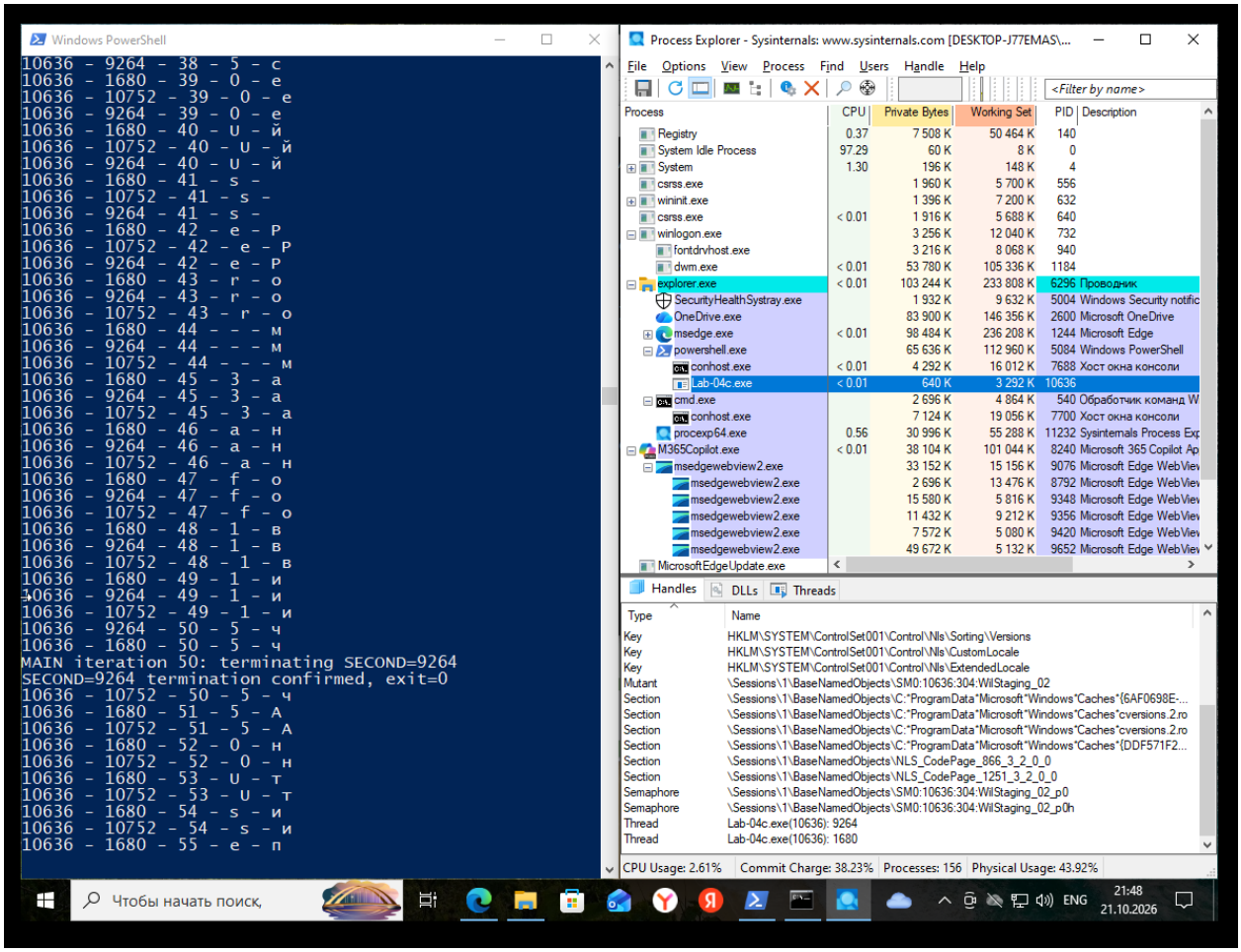


Рисунок 2.10 – Сохранение дескриптора второго дочернего потока после его завершения

Завершение потока не закрывает открытый дескриптор его объекта. Поэтому после TerminateThread объект Thread для TID 9264 остаётся в Handles. В программе CloseHandle вызывается после завершения главного цикла и ожидания дочерних потоков. При принудительном завершении код освобождения буферов внутри дочерней функции не выполняется, поэтому сохранение дескриптора и освобождение выделенной потоком памяти являются разными вопросами.

2.5 Перечисление потоков в Lab 04d

Приложение Lab-04d получает снимок потоков системы функцией CreateToolhelp32Snapshot с флагом TH32CS_SNAPTHREAD. Перед обходом в поле dwSize структуры THREADENTRY32 записывается размер структуры. Первый элемент читается Thread32First, последующие — Thread32Next.

Для каждого элемента проверяется равенство th32OwnerProcessID идентификатору текущего процесса. В консоль выводится th32ThreadID только тех потоков, которые принадлежат приложению. После обхода снимок закрывается функцией CloseHandle. Вывод представлен на рисунке 2.11.

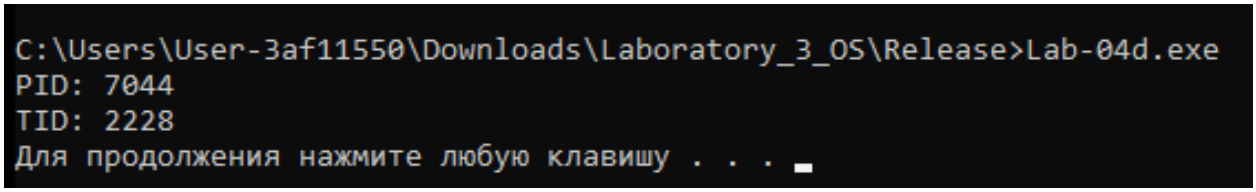


Рисунок 2.11 – Перечень потоков текущего процесса Lab 04d

Программа перечисляет потоки на момент создания снимка. Она не создаёт дополнительные рабочие потоки, поэтому наблюдаемый список отличается от Lab-04a. Снимок представляет состояние в определённый момент и не обновляется автоматически.

2.6 Поиск простых чисел и WinAPI TLS

Функция L4Primes реализована в приложении Lab-04e. Число проверяется перебором возможных делителей от 2 до квадратного корня. Значения не больше 1 исключаются. Для промежуточных результатов каждого потока выделяется отдельный массив, указатель на который записывается в TLS через TlsSetValue.

Общий индекс TLS создаётся TlsAlloc. Для получения локального указателя используется TlsGetValue. После расчёта поток переносит значения из локального буфера в globalPrimes и освобождает буфер. Главный поток после ожидания закрывает дескрипторы и освобождает TLS-индекс [1].

В Lab-04e поиск выполняется тремя потоками на диапазонах [2;3000], [3001;6000] и [6001;9000]. После завершения потоков приложение выводит значение общего счётчика globalIndex. Результат запуска представлен на рисунке 2.12.

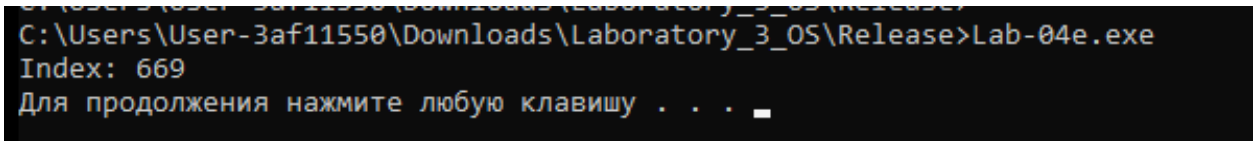


Рисунок 2.12 – Результат приложения Lab 04e с TLS

В диапазоне [2;9000] содержится 669простых числа. На рисунке 2.12 значение globalIndex равно 669, что подтверждает результат.

TLS обеспечивает отдельный промежуточный буфер для каждого потока, но не синхронизирует запись в globalPrimes и изменение globalIndex. Несколько потоков могут использовать одну позицию массива и перезаписывать результаты друг друга. Задержка Sleep(1) между чтением старого значения индекса и его обновлением увеличивает вероятность проявления гонки данных.

2.7 Параметризованный поиск в Lab 04p

Приложение Lab-04p получает три аргумента: число потоков, нижнюю и верхнюю границы диапазона. Диапазон делится на непересекающиеся части. Остаток от деления распределяется по одному дополнительному элементу между первыми потоками. Главный поток ожидает завершения всех созданных потоков, затем сортирует полученные данные функцией qsort.

Каждый рабочий поток вызывает L4Primes для своей части диапазона. Указатель на промежуточный буфер сохраняется через TlsSetValue и получается через TlsGetValue. После поиска поток переносит найденные числа в globalPrimes без синхронизации общего индекса. Главный поток ожидает завершения всех рабочих потоков, проверяет их коды завершения, сортирует сохранённые значения и выводит результат.

На рисунке 2.13 представлены контрольные запуски Lab-04p на диапазоне [2;30] с одним и четырьмя потоками. Эталонная последовательность содержит десять простых чисел: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29.

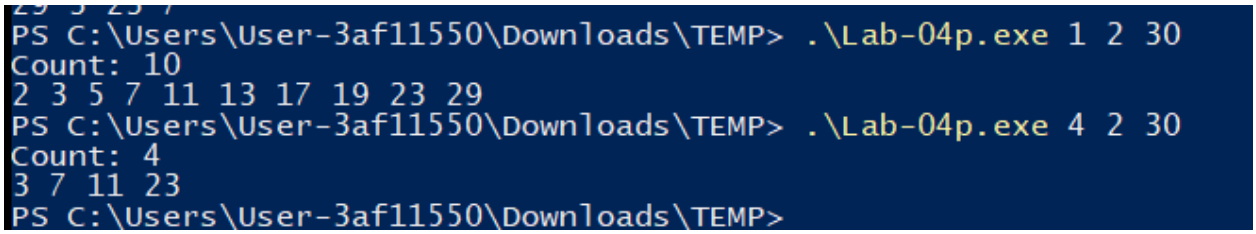


Рисунок 2.13 – Контрольные запуски Lab-04p с одним и четырьмя потоками

При одном потоке получены все десять простых чисел контрольного диапазона. При четырёх потоках сохранены четыре значения: 3, 7, 11 и 23. Неполнота результата связана с несинхронизированной записью в общий массив и изменением globalIndex. Использование TLS изолирует промежуточные буферы, но не устраняет гонку при переносе результатов. Сортировка упорядочивает сохранённые значения, однако не восстанавливает потерянные числа.

2.8 Статический TLS в Lab 04p2

В Lab-04p2 указатель localBuffer и счётчик localCount объявлены с модификатором __declspec(thread). Каждый поток выделяет локальный массив, сохраняет в него простые числа своего диапазона и после завершения поиска переносит значения в общий массив. Динамический TLS-индекс в этом варианте не выделяется.

Число потоков и границы диапазона передаются через аргументы командной строки. Каждый поток сохраняет промежуточные результаты в буфере, указатель на который объявлен с модификатором __declspec(thread). Главный поток ожидает завершения всех рабочих потоков и проверяет их коды завершения. Время измеряется функцией QueryPerformanceCounter от начала создания потоков до окончания ожидания и проверки кодов завершения. Сортировка, вывод чисел и освобождение ресурсов в измеренное время не входят.

На рисунке 2.14 представлены результаты поиска простых чисел в диапазоне [2;30] с одним и четырьмя потоками. Для каждого запуска выведены количество сохранённых чисел, их отсортированная последовательность и измеренное время.

```
PS C:\Users\User-3af11550\Downloads\TEMP> .\Lab-04p2.exe 1 2 30
Count: 10
2 3 5 7 11 13 17 19 23 29
Time: 0.149517 seconds
PS C:\Users\User-3af11550\Downloads\TEMP> .\Lab-04p2.exe 4 2 30
Count: 4
3 7 11 23
Time: 0.054976 seconds
PS C:\Users\User-3af11550\Downloads\TEMP>
```

Рисунок 2.14 – Контрольные запуски Lab-04p2 с одним и четырьмя потоками

При одном потоке получены все десять простых чисел контрольного диапазона. При четырёх потоках сохранены четыре значения: 3, 7, 11 и 23. Статический TLS обеспечивает отдельные промежуточные данные для каждого потока, однако globalPrimes и globalIndex остаются общими. Несинхронизированная запись приводит к неполноте результата. Измеренное время относится к демонстрационной версии с задержкой Sleep(1) при переносе каждого найденного числа и не используется для сравнения производительности.

3 Сравнение производительности

3.1 Условия проведения измерений

Эксперимент выполнен в виртуальной машине с Windows 10 Pro x64 версии 10.0.19045, которой предоставлены восемь виртуальных процессоров. Основной компьютер оснащён процессором AMD Ryzen 7 7735H с восемью физическими ядрами. Число исполнителей K принимало значения 1, 2, 4, 8 и 16.

Для измерений подготовлены отдельные приложения Lab-03d-server-bench, Lab-03d-client-bench, Lab-04p-bench и Lab-04p2-bench в конфигурации Release x64. Клиент процессного варианта выполняет поиск простых чисел вместо проверки взаимной простоты. В обеих потоковых экспериментальных версиях удалена задержка Sleep(1). Демонстрационные приложения раздела 2 сохранены отдельно. В таблице 3.1 экспериментальные потоковые версии обозначены сокращённо как Lab-04p и Lab-04p2.

Во всех вариантах использован диапазон [1;1048576]. Для каждой конфигурации выполнен один прогревочный запуск, не учитываемый в результатах, и три измеряемых запуска. Порядок запуска вариантов циклически изменялся между повторами. В таблице 3.1 приведены медианы трёх измерений.

Полное время выполнения измерялось внешним скриптом средствами System.Diagnostics.Stopwatch непосредственно перед запуском процесса и до его завершения. Измеренный интервал включает создание исполнителей, поиск, сбор результатов, сортировку и вывод. Построение эталонной последовательности, проверка результата и сохранение журналов выполнялись вне измеряемого интервала. Эталон получен решетом Эратосфена и содержит 82025 простых чисел.

Перед основной серией выполнены контрольные запуски всех трёх вариантов на диапазоне [1;30] при K=1 и K=4. Результаты представлены на рисунке 3.1.

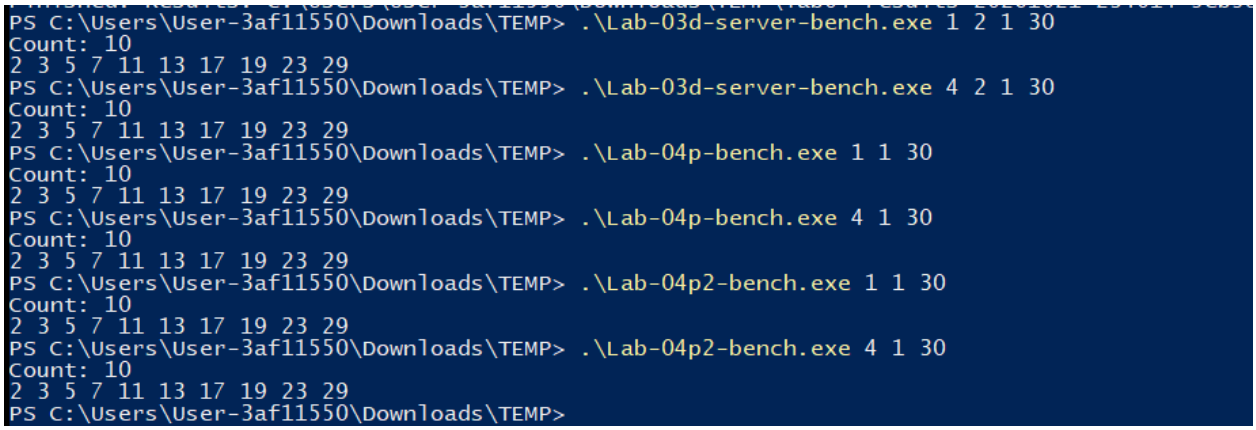


Рисунок 3.1 – Контрольные запуски экспериментальных процессного и потоковых вариантов

Во всех показанных контрольных запусках получена полная отсортированная последовательность из десяти простых чисел: 2, 3, 5, 7, 11, 13, 17, 19, 23 и 29. На малом диапазоне потеря данных в потоковых вариантах не проявилась. Этот результат не гарантирует сохранность данных при других диапазонах и условиях выполнения.

Медианы полного времени выполнения для каждого числа исполнителей представлены в таблице 3.1.

Таблица 3.1 – Медианное полное время выполнения процессного и потоковых вариантов в Windows

K	Процессы ЛР3, с	Lab-04p, с	Lab-04p2, с
1	0,182594	0,140408	0,140247
2	0,136692	0,101895	0,103811
4	0,120910	0,070065	0,074112
8	0,106217	0,056986	0,058847
16	0,122821	0,064049	0,056667

Для каждой конфигурации выполнены три измеряемых запуска после одного прогревочного. В таблице приведены медианы всех трёх измерений. Процессный вариант сохранил полный результат во всех запусках. Для Lab-04p при K=16 полный результат получен в двух из трёх запусков; для Lab-04p2 при K=8 и K=16 — также в двух из трёх. Поэтому приведённые времена несинхронизированных потоковых вариантов необходимо рассматривать совместно с проверкой полноты результата.

3.2 Фиксация результатов и интерпретация

На рисунке 3.2 представлены медианы полного времени выполнения трёх вариантов и результаты проверки полноты данных. Столбцы Processes, WinAPI_TLS и Static_TLS соответствуют процессному варианту, динамическому и статическому TLS. Показатель Complete определяет точное совпадение количества и отсортированной последовательности чисел с эталоном. Значение 3/3 означает совпадение во всех трёх измеряемых запусках.

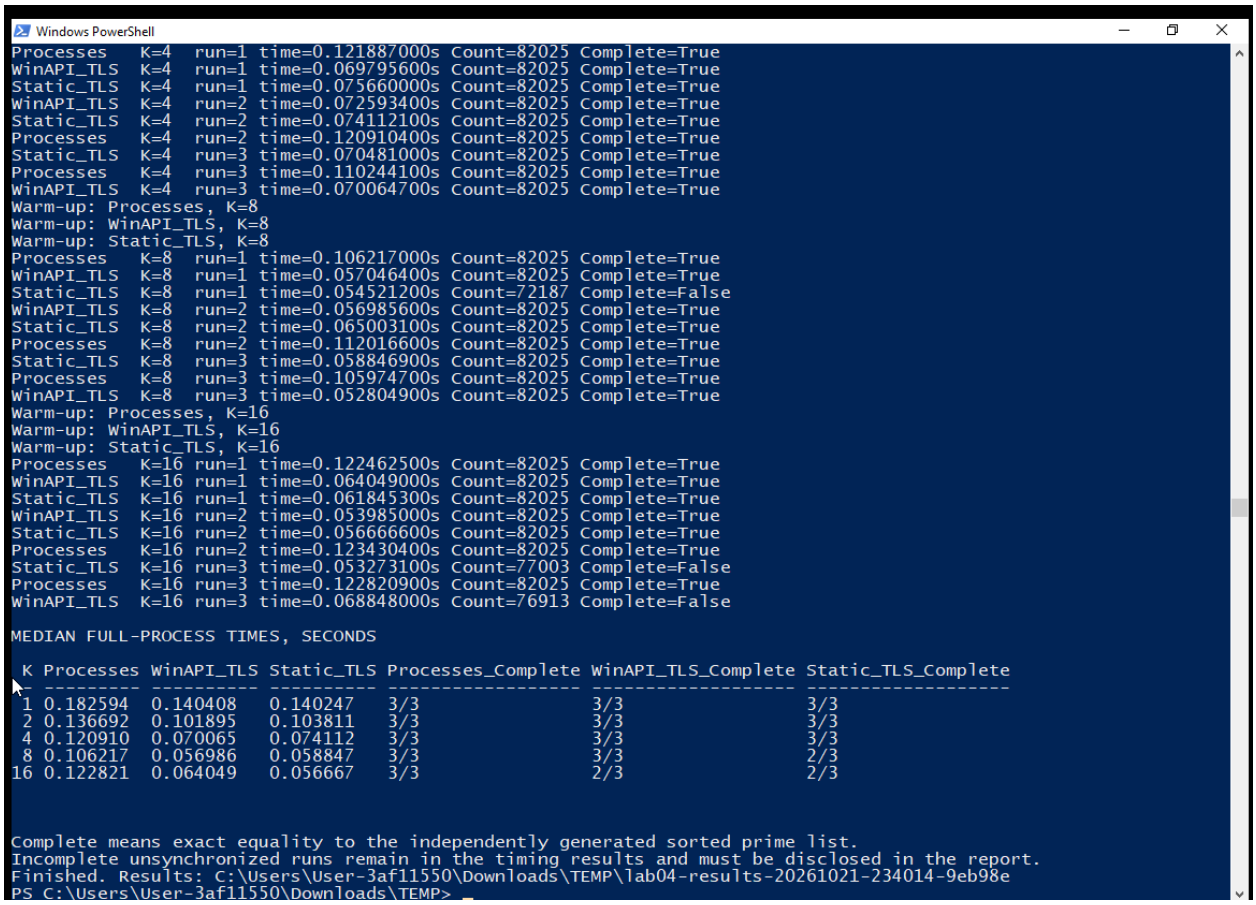


Рисунок 3.2 – Сводные результаты измерений времени выполнения и проверки полноты данных в Windows

Для процессного варианта минимальная медиана времени составила 0,106217 с при K=8. Для Lab-04p минимум составил 0,056986 с при K=8, для Lab-04p2 — 0,056667 с при K=16.

По отношению к запуску с одним исполнителем ускорение составило 1,72 раза для процессного варианта, 2,46 раза для Lab-04p и 2,47 раза для Lab-04p2. Эти значения рассчитаны как отношение медианы времени при K=1 к минимальной медиане соответствующего варианта.

При переходе от K=8 к K=16 время процессного варианта и Lab-04p увеличилось, тогда как для Lab-04p2 несколько уменьшилось. При K=16 число исполнителей превышает восемь виртуальных процессоров, доступных гостевой системе. Поэтому увеличение числа исполнителей не обеспечивает пропорционального уменьшения времени.

Процессный вариант сохранил полный результат во всех измеряемых запусках. В отдельных многопоточных запусках Lab-04p и Lab-04p2 получены неполные результаты из-за несинхронизированной общей записи. Поэтому рассчитанные ускорения характеризуют время исследованных реализаций, но не доказывают ускорение гарантированно корректного накопления всех простых чисел. По одной серии измерений также нельзя утверждать безусловное преимущество одного механизма TLS.

4 Вычисление числа π с использованием волокон

4.1 Приложение Lab 04pi в Windows

В приложении Lab-04pi приближение числа π рассчитывается по ряду Лейбница. Сумма первых M членов задаётся формулой (4.1).

$$\pi_M = 4 \sum_{k=0}^{M-1} \frac{(-1)^k}{2k+1} \quad (4.1)$$

где π_M— полученное приближение числа π;
M — количество обработанных членов ряда;
k — индекс члена ряда, начинающийся с нуля.

Главный поток преобразуется в волокно функцией ConvertThreadToFiber. Число рабочих волокон задаётся аргументом. Каждое получает очередной диапазон из 5000 членов через current_n, добавляет частичную сумму в total_pi и возвращает управление главному волокну через SwitchToFiber. Главный цикл последовательно запускает рабочие волокна [4].

Функцией SetConsoleCtrlHandler регистрируется обработчик CtrlHandler. При получении Ctrl+C обработчик записывает false в атомарную переменную keep_running типа std::atomic_bool. Рабочее волокно проверяет флаг перед началом блока и при вычислении каждого члена ряда. После остановки главный контекст выводит накопленную сумму, умноженную на 4, удаляет рабочие волокна и освобождает массив указателей.

Контрольный запуск выполнен до изменения размера блока, при CHUNK_SIZE = 4000. На рисунке 4.1 показан запуск Lab-04pi с четырьмя рабочими волокнами, результат вычисления после нажатия Ctrl+C и проверка кода завершения средствами PowerShell.

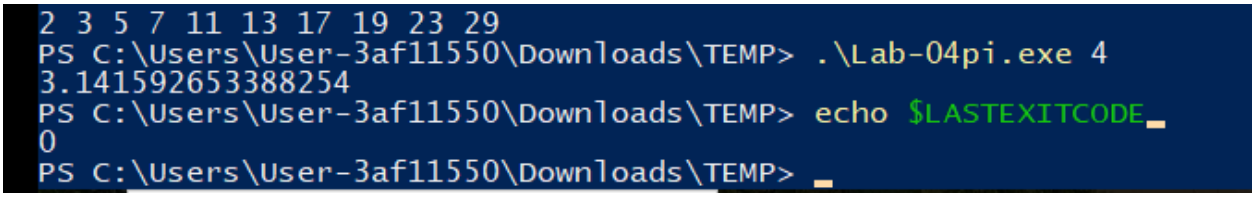


Рисунок 4.1 – Завершение вычисления π по Ctrl+C в Lab-04pi

В показанном запуске получено приближение 3,141592653388254. После вывода результата управление вернулось PowerShell; значение LASTEXITCODE равно 0. Рабочие волокна выполняются последовательно внутри одного системного потока и не обеспечивают параллельного вычисления на нескольких ядрах. Обработчик консоли выполняется в отдельном системном потоке, поэтому запрос остановки передаётся через атомарную переменную.

4.2 Выбор размера частичной суммы

Для выбора размера блока использована отдельная экспериментальная программа Lab-04pi-bench. Вычисления выполнялись четырьмя волокнами при размерах блока 1000, 2000, 4000 и 5000 членов ряда. Остановка автоматически запрашивалась через 10 секунд; фактическая продолжительность измерялась счётчиком QueryPerformanceCounter. Для каждого размера блока выполнен предварительный прогрев продолжительностью одну секунду и три измерительных запуска. Порядок проверки размеров блока изменялся между повторами.

Критерием сравнения выбрана медианная производительность — количество фактически вычисленных членов ряда в секунду. Число обработанных членов и переключений в таблице 4.1 приведено для запуска с медианной производительностью. Учитывались оба направления переключения: от основного волокна к вычислительному и обратно.

Таблица 4.1 – Проверка размеров частичной суммы при четырёх волокнах

Членов в блоке	**Медианная производительность, млн членов/с**	**Обработано членов**	**Переключений**
1000	897,626	8981390294	17962782
2000	923,619	9241020808	9241022
4000	936,076	9373984000	4686994
5000	937,316	9373570800	3749430

При увеличении блока от 1000 до 5000 членов число переключений в выбранных запусках уменьшилось с 17 962 782 до 3 749 430. Это согласуется с более редкой передачей управления между волокнами.

Сводные результаты измерений представлены на рисунке 4.2.

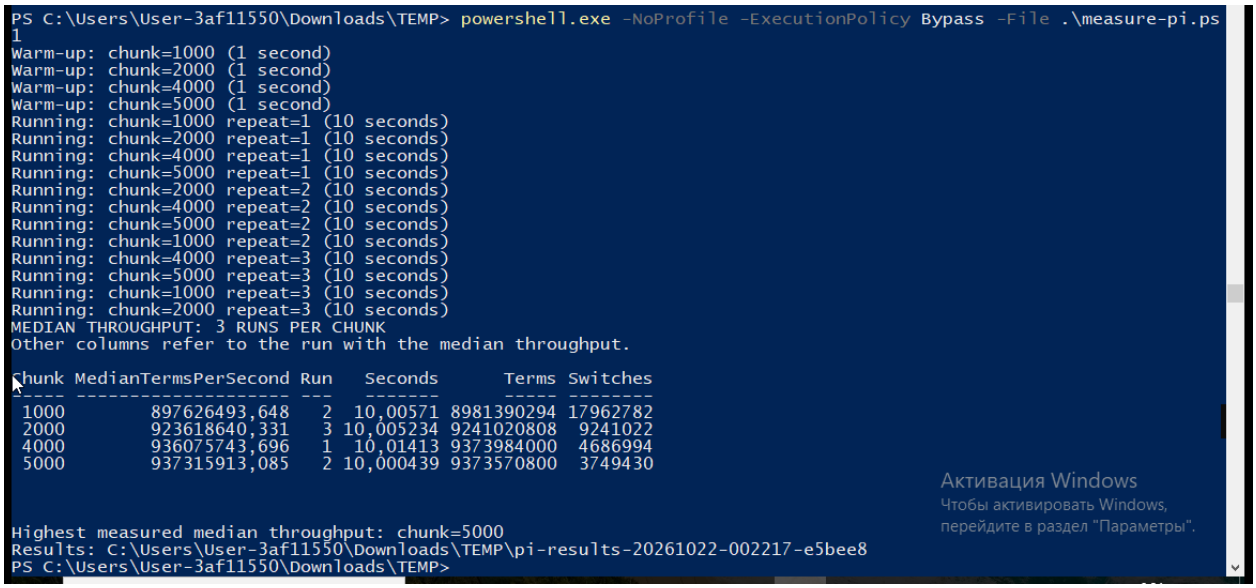


Рисунок 4.2 – Результаты сравнения размеров блока при четырёх волокнах

В проведённой серии максимальная медианная производительность получена при размере блока 5000 членов. Увеличение блока сопровождалось уменьшением количества переключений между волокнами. Для дальнейшего использования выбран размер блока 5000. При этом различие производительности для блоков 4000 и 5000 невелико, поэтому полученные результаты не доказывают устойчивого превосходства блока 5000 в других условиях.

5 Реализация приложений Linux

5.1 Создание потоков и функция Lab_04x

В Linux функция Lab_04x имеет сигнатуру void* Lab_04x(void* arg), соответствующую pthread_create. PID определяется getpid, системный TID — вызовом syscall(SYS_gettid). Для русского текста используется setlocale(LC_ALL, "") и вывод wprintf. Значение задержки usleep находится в диапазоне от 350000 до 450000 мкс.

В Lab-04px создаются два дочерних потока на 50 и 125 итераций. Главный поток вызывает ту же функцию на 100 итераций и ожидает оба потока через pthread_join. В самостоятельном Linux-файле Lab_04x.cpp также реализован запуск трёх потоков с этими значениями.

Вывод программы сохранён в файл Lab-04px-output.txt. После завершения процесса средствами awk подсчитано количество строк каждого потока и определён максимальный номер итерации. Результаты проверки представлены на рисунке 5.1.

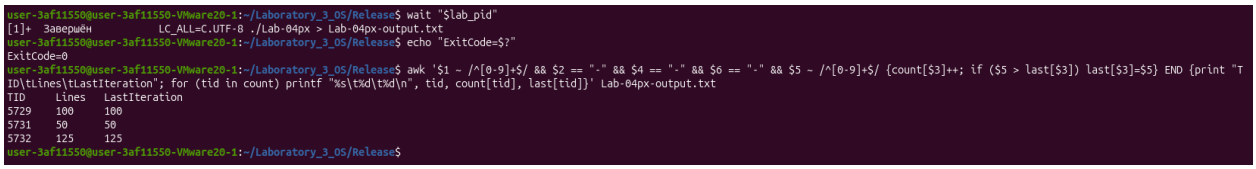


Рисунок 5.1 – Проверка количества итераций потоков Lab-04px в Linux

Программа завершилась с кодом 0. Главный поток с TID 5729 выполнил 100 итераций, дочерний поток с TID 5731 — 50, а поток с TID 5732 — 125. Для каждого потока количество строк совпадает с последним номером итерации.

Сведения о потоках получают командами ps и просмотром /proc. Результат помещается на рисунке 5.2.

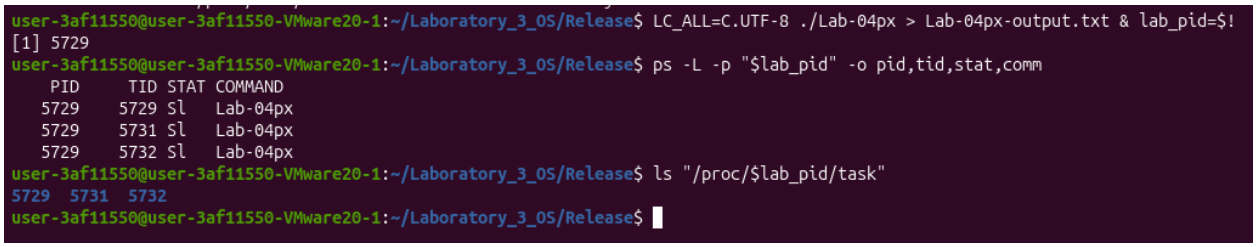


Рисунок 5.2 – Потоки Lab-04px в выводе ps и каталоге /proc/5729/task

Команда ps показывает три потока одного процесса с PID 5729: TID 5729, 5731 и 5732. Те же идентификаторы присутствуют в каталоге /proc/5729/task. В момент наблюдения все три потока находились в состоянии S — прерываемого сна. Дополнительный символ l в столбце STAT обозначает многопоточность процесса.

5.2 Локальное хранение в Lab 04p Linux

Для поиска простых чисел подготовлены два приложения: Lab-04p_Linux с динамическим TLS и Lab-04p2_Linux со статическим TLS. Оба приложения получают количество потоков и границы диапазона через аргументы командной строки. Диапазон разделяется между потоками на части, размеры которых отличаются не более чем на единицу.

В Lab-04p_Linux ключ TLS создаётся функцией pthread_key_create через pthread_once. Каждый поток выделяет структуру LocalData, содержащую указатель на промежуточный буфер и счётчик найденных чисел. Структура связывается с ключом через pthread_setspecific, а доступ к ней выполняется через pthread_getspecific. При завершении потока деструктор destroy_tls освобождает буфер и структуру. После ожидания всех потоков главный поток удаляет ключ.

В Lab-04p2_Linux указатель localBuffer и счётчик localCount объявлены с модификатором __thread, поэтому каждый поток имеет собственные экземпляры этих переменных. Буфер выделяется динамически и освобождается перед возвратом из потоковой функции.

В обоих вариантах найденные числа переносятся в общий массив без синхронизации. Между записью элемента и обновлением общего индекса предусмотрена задержка usleep(1000), повышающая вероятность проявления потери данных. После ожидания созданных потоков главный поток сортирует общий массив и выводит количество сохранённых чисел и сами значения.

Контрольные запуски обоих приложений на диапазоне от 2 до 30 при одном и четырёх потоках представлены на рисунке 5.3.

```
user-3af11550@user-3af11550-VMware20-1:~/Laboratory_3_OS/Release$ printf 'Dynamic TLS: K=1\n'; ./Lab-04p_Linux 1 2 30;
Dynamic TLS: K=1
Count: 10
2 3 5 7 11 13 17 19 23 29
user-3af11550@user-3af11550-VMware20-1:~/Laboratory_3_OS/Release$ printf '\nDynamic TLS: K=4\n'; ./Lab-04p_Linux 4 2 30;

Dynamic TLS: K=4
Count: 4
7 19 23 29
user-3af11550@user-3af11550-VMware20-1:~/Laboratory_3_OS/Release$ printf '\nStatic TLS: K=1\n'; ./Lab-04p2_Linux 1 2 30;

Static TLS: K=1
Count: 10
2 3 5 7 11 13 17 19 23 29
user-3af11550@user-3af11550-VMware20-1:~/Laboratory_3_OS/Release$  printf '\nStatic TLS: K=4\n'; ./Lab-04p2_Linux 4 2 30

Static TLS: K=4
Count: 4
7 19 23 29
user-3af11550@user-3af11550-VMware20-1:~/Laboratory_3_OS/Release$
```

Рисунок 5.3 – Контрольные запуски приложений с динамическим и статическим TLS в Linux

При одном потоке оба приложения вывели все десять простых чисел диапазона: 2, 3, 5, 7, 11, 13, 17, 19, 23 и 29. При четырёх потоках в обоих показанных запусках сохранились только четыре числа: 7, 19, 23 и 29. Следовательно, шесть ожидаемых значений отсутствуют.

Причина потери данных — несинхронизированное обращение к общему массиву и индексу globalIndex. Несколько потоков могут использовать одну позицию записи и перезаписывать результаты друг друга. Независимость промежуточных буферов TLS не защищает общий результат. Совпадение вывода двух вариантов в показанных запусках не гарантирует его повторяемости при следующих запусках.

Для сравнения производительности подготовлены отдельные benchmark-версии приложений. В копиях Lab-04p_Linux и Lab-04p2_Linux удалена искусственная задержка usleep(1000), при этом запись в общий массив осталась несинхронизированной. Процессный вариант Lab-03d-server-bench запускает дочерние процессы Lab-03d-client-bench и получает результаты через канал. Проверка простоты числа во всех трёх вариантах выполняется одинаковым алгоритмом пробного деления.

Перед измерениями выполнены контрольные запуски на диапазоне от 2 до 30 при одном и четырёх исполнителях. Все варианты вывели одинаковый отсортированный список из десяти простых чисел. Результаты представлены на рисунке 5.4. Полнота этих контрольных запусков не гарантирует сохранности данных при других запусках потоковых приложений.



Рисунок 5.4 – Контрольные запуски процессной и потоковых benchmark-версий в Linux

Измерения выполнены в виртуальной машине под управлением Ubuntu 26.04.1 LTS с ядром Linux 7.0.0-31-generic. В гостевой системе доступны восемь логических процессоров; модель процессора — AMD Ryzen 7 7735H with Radeon Graphics. Программы собраны компилятором g++ с оптимизацией -O2.

Диапазон поиска составлял от 1 до 1048576. Количество вычислительных процессов или потоков последовательно принимало значения 1, 2, 4, 8 и 16. Для каждого сочетания приложения и количества исполнителей выполнены один прогревочный и три измерительных запуска. Порядок запуска вариантов изменялся между повторами.

Время измерялось внешним скриптом Python с помощью perf_counter_ns: от момента перед запуском программы до завершения процесса и получения его вывода. Таким образом, учитывались запуск программы, вычисления, сбор результатов, сортировка и вывод. Подготовка эталона, проверка результата и сохранение журналов выполнялись вне измеряемого интервала.

Корректность проверялась сравнением полного отсортированного списка с независимым эталоном, сформированным решетом Эратосфена. Эталон содержит 82025 простых чисел. В таблице 5.1 приведены медианы трёх измерений, включая запуски с неполным результатом.

Таблица 5.1 – Медианное время выполнения benchmark-версий в Linux

K	**Процессный вариант, с**	**Динамический TLS, с**	**Статический TLS, с**
1	0,217239	0,209994	0,213621
2	0,140188	0,135712	0,134087
4	0,083830	0,083878	0,080729
8	0,054395	0,059139	0,062642
16	0,061073	0,050779	0,050272

Для динамического TLS при K = 16 полный результат получен только в двух из трёх измерительных запусков. Во всех остальных сочетаниях варианта и количества исполнителей полное совпадение с эталоном получено в трёх из трёх запусков. Сводные результаты измерений и проверки полноты данных представлены на рисунке 5.5.

Сводные показатели времени и полноты результата приведены на рисунке 5.5.

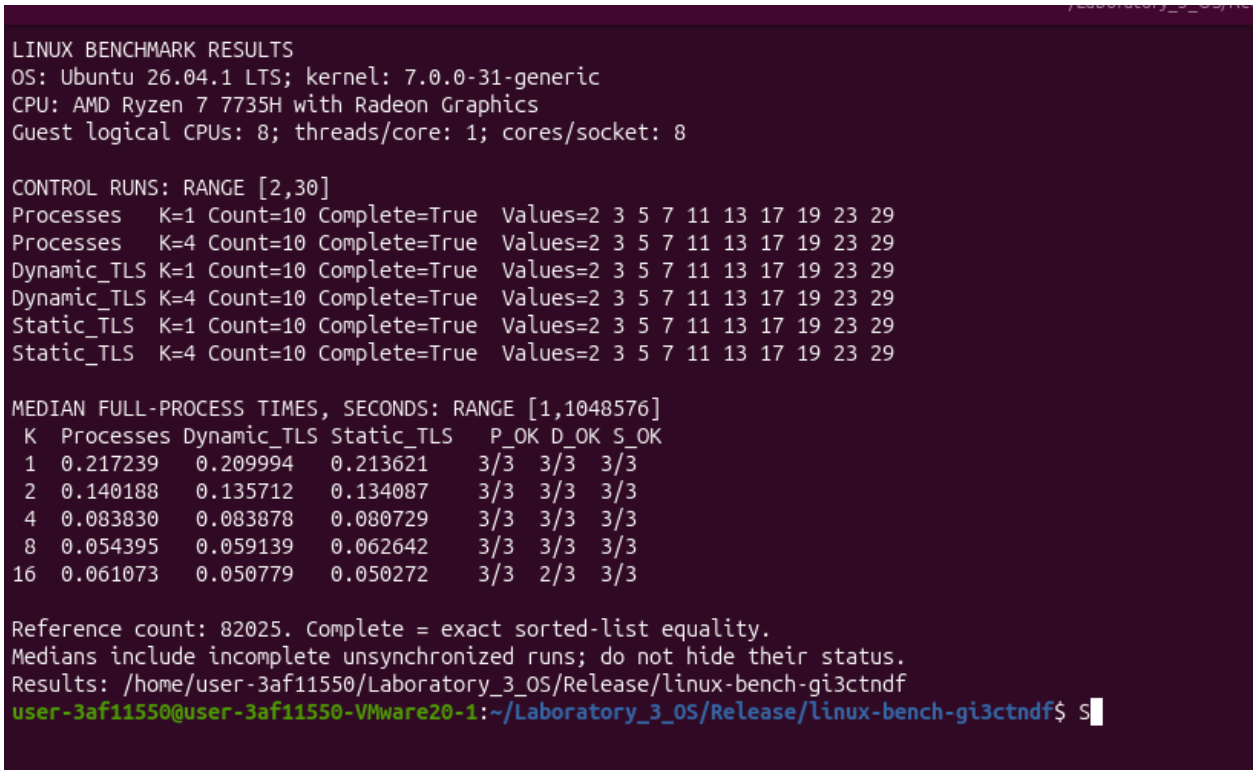


Рисунок 5.5 – Результаты измерения времени и проверки полноты данных в Linux

В проведённой серии минимальное медианное время процессного варианта получено при K = 8 и составило 0,054395 с. Увеличение количества процессов до 16 привело к росту времени выполнения. Для динамического и статического TLS минимальные медианы получены при K = 16: 0,050779 и 0,050272 с соответственно.

При K = 16 количество исполнителей превышает число логических процессоров гостевой системы. Поэтому увеличение количества процессов или потоков не означает увеличения доступного аппаратного параллелизма. Полученные зависимости относятся к данной серии измерений и конфигурации виртуальной машины.

Неполный результат одного запуска динамического TLS подтверждает, что удаление искусственной задержки не устраняет гонку данных. Полученные времена характеризуют выполнение несинхронизированных реализаций и не должны интерпретироваться как гарантированное ускорение корректного вычисления.

5.3 Вычисление π через пользовательские контексты

В Lab-04pi_Linux волокна реализованы с помощью ucontext_t. Для каждого рабочего контекста вызывается getcontext, выделяется стек размером 64 КБ и назначается функция fiber_func через makecontext. В поле uc_link записан указатель на main_ctx. Главный цикл выбирает current_fiber и переключается к нему через swapcontext [8].

Рабочий контекст обрабатывает очередной блок из 5000 членов ряда Лейбница по формуле (4.1). Частичная сумма добавляется к total_pi, затем выполняется обратное переключение к main_ctx. Несколько контекстов последовательно используют один поток операционной системы.

Для SIGINT установлен обработчик sigaction. Он изменяет только keep_running, объявленный volatile sig_atomic_t. Печать результата и освобождение памяти выполняются после выхода из вычислительного цикла. На штатном пути освобождаются стеки всех контекстов, массив stacks и массив fiber_ctxs.

Программа запущена командой ./Lab-04pi_Linux 4 с четырьмя рабочими контекстами. После нажатия Ctrl+C получено приближение π = 3,141592653595123. Код завершения процесса равен 0. Контрольный запуск выполнен при размере блока 4000 членов. Результат представлен на рисунке 5.6.

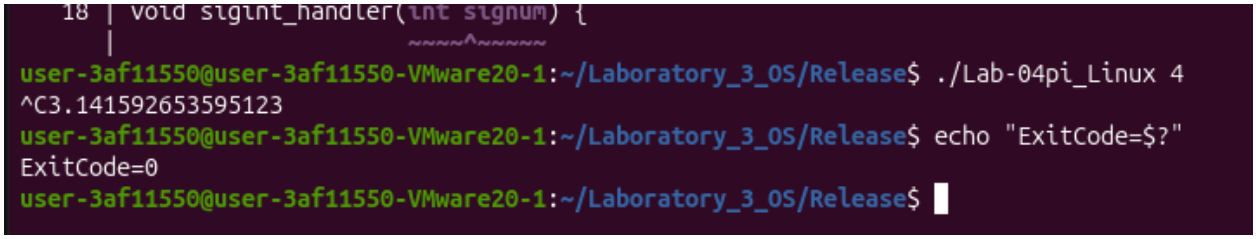


Рисунок 5.6 – Завершение вычисления π по Ctrl+C в Linux

Механизм ucontext обеспечивает кооперативное переключение на уровне пользователя. Он не заменяет pthread_create для параллельного расчёта на нескольких ядрах. Функции семейства ucontext доступны в рассматриваемой Linux-среде, но не относятся к современному переносимому интерфейсу POSIX [8].

Для выбора размера блока в Linux подготовлена отдельная экспериментальная программа Lab-04pi_Linux-bench. Проверены блоки из 1000, 2000, 4000 и 5000 членов ряда при четырёх рабочих контекстах. Вычисления во всех контекстах выполнялись последовательно в одном потоке ОС. Дополнительный служебный поток запрашивал остановку через 10 секунд посредством атомарного флага.

Для каждого размера блока выполнены один прогревочный запуск продолжительностью одну секунду и три измерительных запуска. Порядок проверки размеров блока изменялся между повторами. Фактическая продолжительность измерялась с помощью clock_gettime с часами CLOCK_MONOTONIC.

Критерием сравнения выбрана медианная производительность — количество фактически вычисленных членов ряда в секунду. В таблице 5.2 число обработанных членов и переключений приведено для запуска с медианной производительностью. Учитывались явные переключения swapcontext в обоих направлениях.

Таблица 5.2 – Влияние размера блока на производительность вычисления π в Linux

Членов в блоке	**Медианная производительность, млн членов/с**	**Обработано членов**	**Переключений**
1000	670,883	6708900000	13417801
2000	785,060	7850757900	7850758
4000	865,566	8655750056	4327876
5000	879,576	8795853933	3518342

Среди проверенных размеров блока наибольшая медианная производительность получена для 5000 членов. Для этого размера в выбранном запуске зарегистрировано 3 518 342 переключения.

Результаты сравнения размеров блока представлены на рисунке 5.7.

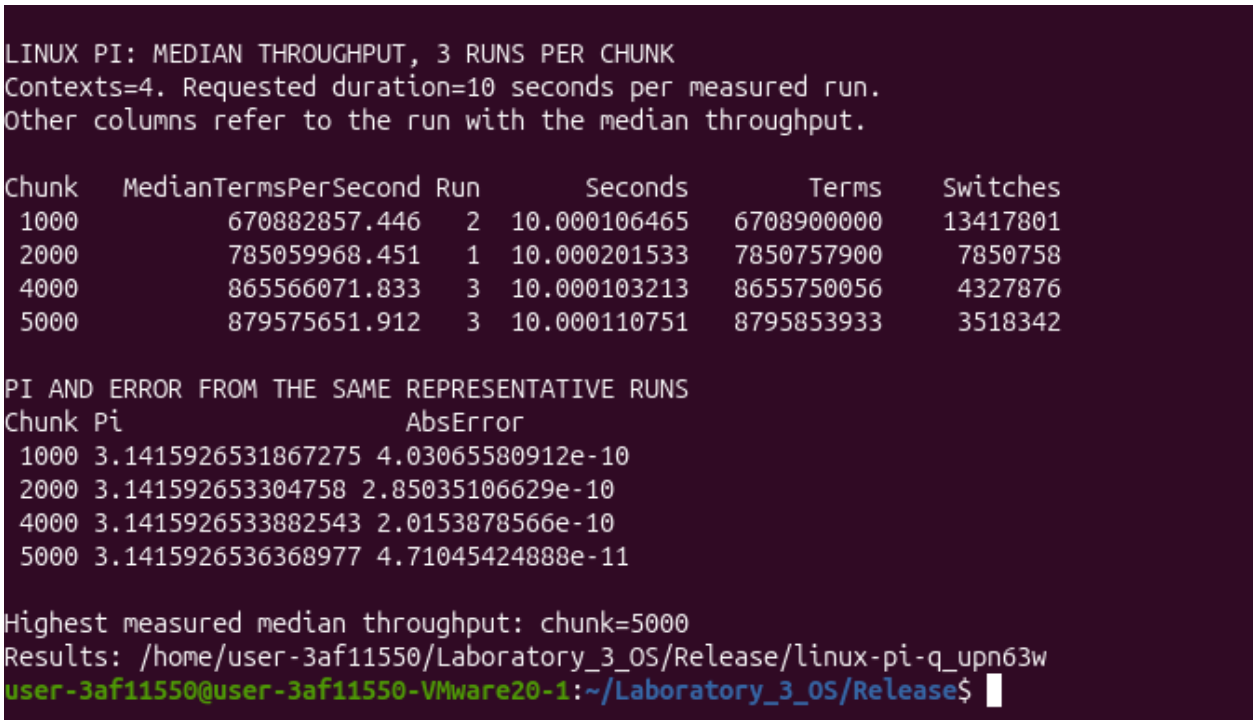


Рисунок 5.7 – Результаты выбора размера блока для вычисления π в Linux

В проведённой серии максимальная медианная производительность получена при размере блока 5000 членов и составила 879,576 млн членов/с. Увеличение блока сопровождалось уменьшением количества переключений. Для дальнейшего использования выбран размер блока 5000.

В запуске с медианной производительностью для выбранного блока получено приближение π = 3,1415926536368977. Абсолютное отклонение от эталонного значения двойной точности составило приблизительно $4{,}71 \cdot 10^{-11}$. Выбор размера блока выполнен по производительности, а не по отклонению отдельного результата. Вывод относится к исследованным размерам блока и данной конфигурации виртуальной машины.

Заключение

В лабораторной работе исследованы создание потоков и управление их выполнением в Windows и Linux. Проверены идентификаторы процессов и потоков, заданное количество итераций и ожидание завершения дочерних потоков. Наблюдение выполнялось средствами Process Explorer, ps и файловой системы /proc.

В Windows рассмотрены приостановка, возобновление и принудительное завершение потоков. Установлено, что завершение потока не означает автоматического закрытия открытого дескриптора. Принудительное завершение также не обеспечивает выполнения кода освобождения локальных ресурсов.

Поиск простых чисел реализован с динамическим и статическим TLS. Контрольные запуски продемонстрировали потерю данных при несинхронизированной записи в общий массив. Независимость локальных буферов не защищает общий индекс записи, а последующая сортировка не восстанавливает потерянные значения.

Для сравнения производительности подготовлены отдельные benchmark-версии без искусственных задержек. В Windows и Linux выполнены серии измерений при количестве исполнителей 1, 2, 4, 8 и 16. Результаты проверялись сравнением полного отсортированного списка с независимым эталоном. Некоторые многопоточные запуски дали неполный результат, поэтому их время нельзя рассматривать как гарантированное ускорение корректного вычисления. Увеличение количества исполнителей не во всех случаях уменьшало время выполнения.

Вычисление π выполнено с использованием волокон Windows и пользовательских контекстов Linux. Проверена остановка по Ctrl+C с выводом накопленного приближения. В отдельных экспериментах сопоставлены размеры блока 1000, 2000, 4000 и 5000 членов ряда. В обеих системах наибольшая медианная производительность в проведённых сериях получена при размере блока 5000, который выбран для основных программ. Этот результат относится к исследованным условиям и не устанавливает универсально оптимальный размер блока.

Список использованных источников

1. Thread Local Storage. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/thread-local-storage (дата обращения: 14.10.2026).

2. SuspendThread function. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-suspendthread (дата обращения: 14.10.2026).

3. Terminating a Thread. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/terminating-a-thread (дата обращения: 14.10.2026).

4. Using Fibers. Microsoft Learn. URL: https://learn.microsoft.com/en-gb/windows/win32/procthread/using-fibers (дата обращения: 14.10.2026).

5. pthread_once. POSIX Programmer’s Manual. URL: https://www.man7.org/linux/man-pages/man3/pthread_once.3p.html (дата обращения: 14.10.2026).

6. pthread_key_create. Linux manual page. URL: https://www.man7.org/linux/man-pages/man3/pthread_setspecific.3.html (дата обращения: 14.10.2026).

7. pthreads. Linux manual page. URL: https://www.man7.org/linux/man-pages/man7/pthreads.7.html (дата обращения: 14.10.2026).

8. makecontext, swapcontext. Linux manual page. URL: https://www.man7.org/linux/man-pages/man3/swapcontext.3.html (дата обращения: 14.10.2026).

9. proc_pid_task. Linux manual page. URL: https://www.man7.org/linux/man-pages/man5/proc_pid_task.5.html (дата обращения: 14.10.2026).

Приложение А Основные фрагменты программ

Ниже приведены фрагменты исходных файлов, определяющие создание потоков, использование TLS и переключение контекстов. Полные исходные программы находятся в каталогах Lab_04x и Linux.

Листинг А.1 – Обёртка потоковой функции Windows

```
DWORD WINAPI ThreadWrapper(LPVOID lpParam) {
    int iterations = *((int*)lpParam);
    Lab_04x(iterations);
    return 0;
}
```

Источник фрагмента — файл Lab-04a.cpp.

Обёртка получает количество итераций через указатель int* и вызывает Lab_04x. Возврат из ThreadWrapper завершает дочерний поток.

Отбор потоков по идентификатору процесса и закрытие снимка показаны в листинге А.2.

Листинг А.2 – Перечисление потоков текущего процесса

```
do {
    if (te32.th32OwnerProcessID == currentPid) {
        wprintf(L"TID: %lu\n", te32.th32ThreadID);
    }
} while (Thread32Next(hThreadSnap, &te32));
CloseHandle(hThreadSnap);
```

Источник фрагмента — файл Lab-04d.cpp.

Сравнение идентификатора владельца со значением currentPid отбирает потоки текущего процесса из общего снимка.

Локальные данные потоков Windows и Linux

В листингах А.3 и А.4 показаны доступ к промежуточному буферу WinAPI TLS и однократная инициализация ключа TLS в Linux.

Листинг А.3 – Использование буфера WinAPI TLS

```
int localCount = 0;
TlsSetValue(tlsIndex, localBuffer);
for (int i = start; i <= end; i++) {
    if (isPrime(i)) {
        int* buf = (int*)TlsGetValue(tlsIndex);
        buf[localCount] = i;
        localCount++;
    }
}
int* finalBuf = (int*)TlsGetValue(tlsIndex);
for (int i = 0; i < localCount; i++) {
    globalPrimes[globalIndex] = finalBuf[i];
    int temp = globalIndex;
    Sleep(1);
    globalIndex = temp + 1;
}
free(finalBuf);
```

Источник фрагмента — файл Lab-04e.cpp.

Буфер, возвращаемый TlsGetValue, принадлежит текущему потоку. Перенос в globalPrimes выполняется через общий индекс и не синхронизируется.

В листинге А.4 приведены структура локальных данных, создание ключа TLS и функция освобождения буфера при завершении потока.

Листинг А.4 – Однократная инициализация TLS в Linux

```
struct LocalData {
    int* buffer;
    int count;
};

pthread_once_t once_control = PTHREAD_ONCE_INIT;
pthread_key_t tlsKey;
int tls_error = 0;

void destroy_tls(void* pointer) {
    LocalData* data = static_cast<LocalData*>(pointer);
    if (data) {
        free(data->buffer);
        free(data);
    }
}

void init_tls() {
    tls_error = pthread_key_create(&tlsKey, destroy_tls);
}
```

Источник фрагмента — файл Lab-04p_Linux.cpp.

Функция init_tls вызывается через pthread_once перед созданием рабочих потоков. Каждый поток связывает собственную структуру LocalData с ключом TLS. При завершении потока деструктор destroy_tls освобождает промежуточный буфер и структуру. Переменные с модификатором __thread в данном варианте не используются.

Расчёт блока ряда в волокне Windows

В листинге А.5 приведён расчёт частичной суммы и возврат управления главному волокну.

Листинг А.5 – Функция FiberFunc

```
void WINAPI FiberFunc(LPVOID lpParameter) {
   while (keep_running.load(std::memory_order_relaxed)) {
      long long start = current_n;
      current_n += CHUNK_SIZE;
      double partial_sum = 0.0;
      for (long long i = start; i < start + CHUNK_SIZE; i++) {
         if (!keep_running.load(std::memory_order_relaxed)) break;
         double term = 1.0 / (2.0 * i + 1.0);
         if (i % 2 != 0) {
            term = -term;
         }
         partial_sum += term;
      }
      total_pi += partial_sum;
      SwitchToFiber(main_fiber);
   }
   SwitchToFiber(main_fiber);
}
```

Источник фрагмента — файл Lab-04pi.cpp.

После добавления частичной суммы управление возвращается главному волокну. Следующий вызов продолжает выполнение функции после SwitchToFiber и выдаёт очередной блок индексов.

Создание и переключение контекстов Linux

В листинге А.6 показаны настройка стеков рабочих контекстов и последовательное переключение к ним из главного контекста.

Листинг А.6 – Рабочие контексты ucontext

```
for (int i = 0; i < num_fibers; i++) {
    getcontext(&fiber_ctxs[i]);
    stacks[i] = (char*)malloc(64 * 1024);
    fiber_ctxs[i].uc_stack.ss_sp = stacks[i];
    fiber_ctxs[i].uc_stack.ss_size = 64 * 1024;
    fiber_ctxs[i].uc_link = &main_ctx;
    makecontext(&fiber_ctxs[i], fiber_func, 0);
}
while (keep_running) {
    for (int i = 0; i < num_fibers; i++) {
        if (!keep_running) break;
        current_fiber = i;
        swapcontext(&main_ctx, &fiber_ctxs[i]);
    }
}
```

Источник фрагмента — файл Lab-04pi_Linux.cpp.

Каждый контекст получает отдельный стек и функцию fiber_func. Сохранение и восстановление состояний выполняются swapcontext, а номер активного контекста передаётся через current_fiber.